

LAB01 Data Inspection and Cleaning

This notebook works directly with the existing Python environment in VS Code. It investigates the two CSV files in `LAB01/Datasets`, documents what is inside them, and applies the cleaning steps needed for the lab tasks.

The notebook is organized around the five lab requirements:

1. Profile both files
2. Treat missing values with column-specific decisions
3. Normalize text fields and remove duplicates
4. Study the AQI distribution
5. Detect and handle extreme AQI values

```
In [1]: from pathlib import Path
import sys
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings('ignore')

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 120)
sns.set_theme(style='whitegrid')

print('Python executable:', sys.executable)
print('Python version:', sys.version.split()[0])
```

Python executable: c:\Users\amogh\AppData\Local\Programs\Python\Python312\python.exe

Python version: 3.12.9

```
In [3]: DATA_DIR = Path(r'e:\College\MCA\Trimester 4\ML Labs\LabProgs\LAB01\Datasets')
CROP_PATH = DATA_DIR / 'crop_production.csv'
CITY_PATH = DATA_DIR / 'city_day.csv'
OUTPUT_DIR = Path(r'e:\College\MCA\Trimester 4\ML Labs\LabProgs\LAB01') / 'outputs'
OUTPUT_DIR.mkdir(exist_ok=True)

print('Data directory:', DATA_DIR.resolve())
print('Crop file exists:', CROP_PATH.exists())
print('City file exists:', CITY_PATH.exists())
print('Output directory:', OUTPUT_DIR.resolve())
```

Data directory: E:\College\MCA\Trimester 4\ML Labs\LabProgs\LAB01\Datasets

Crop file exists: True

City file exists: True

Output directory: E:\College\MCA\Trimester 4\ML Labs\LabProgs\LAB01\outputs

```
In [4]: crop_df = pd.read_csv(CROP_PATH)
city_df = pd.read_csv(CITY_PATH)
```

```

print('Crop shape:', crop_df.shape)
print('City shape:', city_df.shape)
print('\nCrop columns:\n', crop_df.columns.tolist())
print('\nCity columns:\n', city_df.columns.tolist())

print('\nCrop sample:')
display(crop_df.head())
print('\nCity sample:')
display(city_df.head())

```

Crop shape: (246091, 7)

City shape: (29531, 16)

Crop columns:

['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop', 'Area', 'Production']

City columns:

['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket']

Crop sample:

	State_Name	District_Name	Crop_Year	Season	Crop	Area	Production
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Arecanut	1254.0	2000.0
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Other Kharif pulses	2.0	1.0
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Rice	102.0	321.0
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Banana	176.0	641.0
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Cashewnut	720.0	165.0

City sample:

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	133.36	
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	34.06	
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	30.70	
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	36.08	
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	39.31	



Task 1. Structured data profile

A data scientist should know the shape, columns, data types, null counts, duplicate counts, and whether the file contains suspicious categorical values before trusting it for modeling.

```
In [5]: def profile_frame(df: pd.DataFrame, name: str) -> dict:
        text_cols = df.select_dtypes(include='object').columns.tolist()
        numeric_cols = df.select_dtypes(include='number').columns.tolist()
        summary = {
            'dataset': name,
            'shape': df.shape,
            'columns': list(df.columns),
            'dtypes': df.dtypes.astype(str).to_dict(),
            'missing_counts': df.isna().sum().to_dict(),
            'duplicate_rows': int(df.duplicated().sum()),
            'numeric_columns': numeric_cols,
            'categorical_columns': text_cols,
        }
        return summary

        crop_profile = profile_frame(crop_df, 'crop_production')
        city_profile = profile_frame(city_df, 'city_day')

        for profile in (crop_profile, city_profile):
            print(f"\n=== {profile['dataset']} ===")
            print('Shape:', profile['shape'])
            print('Duplicate rows:', profile['duplicate_rows'])
            print('Numeric columns:', profile['numeric_columns'])
            print('Categorical columns:', profile['categorical_columns'])
            print('Missing counts:')
            display(pd.Series(profile['missing_counts']).sort_values(ascending=False).to
            print('Dtypes:')
            display(pd.Series(profile['dtypes']).to_frame('dtype'))

        print('Initial concern 1: city_day has several columns with substantial nulls, s
        print('Initial concern 2: crop_production has text fields with trailing spaces,
```

```
=== crop_production ===
Shape: (246091, 7)
Duplicate rows: 0
Numeric columns: ['Crop_Year', 'Area', 'Production']
Categorical columns: ['State_Name', 'District_Name', 'Season', 'Crop']
Missing counts:
```

missing_values	
Production	3730
District_Name	0
State_Name	0
Crop_Year	0
Season	0
Crop	0
Area	0

Dtypes:

dtype	
State_Name	str
District_Name	str
Crop_Year	int64
Season	str
Crop	str
Area	float64
Production	float64

```
=== city_day ===
Shape: (29531, 16)
Duplicate rows: 0
Numeric columns: ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
                  'Benzene', 'Toluene', 'Xylene', 'AQI']
Categorical columns: ['City', 'Date', 'AQI_Bucket']
Missing counts:
```

missing_values	
Xylene	18109
PM10	11140
NH3	10328
Toluene	8041
Benzene	5623
AQI	4681
AQI_Bucket	4681
PM2.5	4598
NOx	4185
O3	4022
SO2	3854
NO2	3585
NO	3582
CO	2059
Date	0
City	0

```
Dtypes:
```

	dtype
City	str
Date	str
PM2.5	float64
PM10	float64
NO	float64
NO2	float64
NOx	float64
NH3	float64
CO	float64
SO2	float64
O3	float64
Benzene	float64
Toluene	float64
Xylene	float64
AQI	float64
AQI_Bucket	str

Initial concern 1: city_day has several columns with substantial nulls, so blanket row deletion would waste too much data.

Initial concern 2: crop_production has text fields with trailing spaces, which can break grouping and joins if not normalized.

Task 2. Missing-value treatment strategy

The missing-value policy below is column specific:

- numeric measurement columns are imputed with the median because it is safer than the mean for skewed pollution and production data
- target-like columns are dropped row-wise when the missingness is limited and synthetic values would be misleading
- redundant or secondary columns with too many holes are removed instead of forcing weak imputation

```
In [6]: def missing_report(df: pd.DataFrame) -> pd.DataFrame:
        report = pd.DataFrame({
            'missing_count': df.isna().sum(),
            'missing_pct': (df.isna().mean() * 100).round(2),
            'dtype': df.dtypes.astype(str),
        })
        return report[report['missing_count'] > 0].sort_values('missing_count', ascending=True)

def treat_missing_values(df: pd.DataFrame, dataset_name: str) -> tuple[pd.DataFrame, pd.DataFrame]:
```

```

cleaned = df.copy()
report = missing_report(cleaned)
decisions = []

for column, row in report.iterrows():
    pct = float(row['missing_pct'])
    dtype = row['dtype']
    if column == 'AQI':
        affected = int(cleaned[column].isna().sum())
        cleaned = cleaned.dropna(subset=['AQI'])
        decisions.append((column, affected, 'drop affected rows', 'AQI is th
    elif column == 'AQI_Bucket':
        affected = int(cleaned[column].isna().sum())
        cleaned = cleaned.drop(columns=['AQI_Bucket'])
        decisions.append((column, affected, 'drop column', 'AQI_Bucket is a
    elif column == 'Production' and dataset_name == 'crop_production':
        affected = int(cleaned[column].isna().sum())
        cleaned = cleaned.dropna(subset=[column])
        decisions.append((column, affected, 'drop affected rows', 'Productio
    elif dtype == 'object':
        mode_value = cleaned[column].mode(dropna=True)
        fill_value = mode_value.iloc[0] if not mode_value.empty else 'Unknow
        cleaned[column] = cleaned[column].fillna(fill_value)
        decisions.append((column, int(df[column].isna().sum()), 'impute with
    else:
        if pct >= 25:
            cleaned = cleaned.drop(columns=[column])
            decisions.append((column, int(df[column].isna().sum()), 'drop co
        else:
            if 'City' in cleaned.columns and cleaned['City'].notna().any():
                cleaned[column] = cleaned.groupby('City')[column].transform(
            if cleaned[column].isna().any():
                cleaned[column] = cleaned[column].fillna(cleaned[column].med
            decisions.append((column, int(df[column].isna().sum()), 'impute

decision_table = pd.DataFrame(decisions, columns=['column', 'missing_before'
return cleaned, decision_table

crop_clean, crop_missing_decisions = treat_missing_values(crop_df, 'crop_product
city_clean, city_missing_decisions = treat_missing_values(city_df, 'city_day')

print('Crop missing-value decisions:')
display(crop_missing_decisions)
print('City missing-value decisions:')
display(city_missing_decisions)

print('Null counts before cleaning - crop_production:')
display(missing_report(crop_df))
print('Null counts after cleaning - crop_production:')
display(missing_report(crop_clean))
print('Null counts before cleaning - city_day:')
display(missing_report(city_df))
print('Null counts after cleaning - city_day:')
display(missing_report(city_clean))

```

Crop missing-value decisions:

	column	missing_before	action	reason
0	Production	3730	drop affected rows	Production is the outcome variable for the cro...

City missing-value decisions:

	column	missing_before	action	reason
0	Xylene	18109	drop column	Xylene has 61.32% missingness, which is too sp...
1	PM10	11140	drop column	PM10 has 37.72% missingness, which is too spar...
2	NH3	10328	drop column	NH3 has 34.97% missingness, which is too spars...
3	Toluene	8041	drop column	Toluene has 27.23% missingness, which is too s...
4	Benzene	5623	impute with median	Benzene is numeric; the median is robust to sk...
5	AQI	4681	drop affected rows	AQI is the key analysis variable; imputing it ...
6	AQI_Bucket	0	drop column	AQI_Bucket is a derived label and can be recon...
7	PM2.5	4598	impute with median	PM2.5 is numeric; the median is robust to skew...
8	NOx	4185	impute with median	NOx is numeric; the median is robust to skew a...
9	O3	4022	impute with median	O3 is numeric; the median is robust to skew an...
10	SO2	3854	impute with median	SO2 is numeric; the median is robust to skew a...
11	NO2	3585	impute with median	NO2 is numeric; the median is robust to skew a...
12	NO	3582	impute with median	NO is numeric; the median is robust to skew an...
13	CO	2059	impute with median	CO is numeric; the median is robust to skew an...

Null counts before cleaning - crop_production:

	missing_count	missing_pct	dtype
Production	3730	1.52	float64

Null counts after cleaning - crop_production:

missing_count	missing_pct	dtype
---------------	-------------	-------

Null counts before cleaning - city_day:

	missing_count	missing_pct	dtype
Xylene	18109	61.32	float64
PM10	11140	37.72	float64
NH3	10328	34.97	float64
Toluene	8041	27.23	float64
Benzene	5623	19.04	float64
AQI	4681	15.85	float64
AQI_Bucket	4681	15.85	str
PM2.5	4598	15.57	float64
NOx	4185	14.17	float64
O3	4022	13.62	float64
SO2	3854	13.05	float64
NO2	3585	12.14	float64
NO	3582	12.13	float64
CO	2059	6.97	float64

Null counts after cleaning - city_day:

missing_count	missing_pct	dtype
---------------	-------------	-------

Task 3. State-name normalization and duplicate removal

The lab prompt refers to a `State` key, but in these files the state-like field is `State_Name` in the crop dataset. The code below normalizes all text fields, lists the raw variants that collapse to the same cleaned value, and removes duplicate records so the file is merge-ready.

```
In [7]: def normalize_text(value):
        if pd.isna(value):
            return value
        return ' '.join(str(value).split())

def normalize_text_columns(df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.copy()
    for column in cleaned.select_dtypes(include='object').columns:
        cleaned[column] = cleaned[column].map(normalize_text)
    return cleaned

def text_variants(df: pd.DataFrame, column: str) -> pd.DataFrame:
    raw = df[column].dropna().astype(str)
    normalized = raw.map(normalize_text)
    variants = (
```



```

        pd.DataFrame({'raw': raw, 'normalized': normalized})
        .groupby('normalized')['raw']
        .agg(lambda s: sorted(set(s)))
        .reset_index()
    )
    variants['variant_count'] = variants['raw'].map(len)
    variants = variants[variants['variant_count'] > 1].sort_values('variant_count', ascending=False)
    return variants

crop_text_variants = text_variants(crop_clean, 'State_Name') if 'State_Name' in crop_clean.columns else pd.DataFrame()
print('State-name variants found in crop_production:')
display(crop_text_variants if not crop_text_variants.empty else pd.DataFrame({'message': 'No multi-spelling state-name variants found after deduplication.'}))

crop_merge_ready = normalize_text_columns(crop_clean)
city_merge_ready = normalize_text_columns(city_clean)

crop_before = len(crop_merge_ready)
city_before = len(city_merge_ready)

crop_merge_ready = crop_merge_ready.drop_duplicates()
city_merge_ready = city_merge_ready.drop_duplicates()

crop_key_cols = ['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop']
if all(col in crop_merge_ready.columns for col in crop_key_cols):
    crop_merge_ready = crop_merge_ready.drop_duplicates(subset=crop_key_cols)

if all(col in city_merge_ready.columns for col in ['City', 'Date']):
    city_merge_ready = city_merge_ready.drop_duplicates(subset=['City', 'Date'])

print(f'Crop rows before: {crop_before}, after duplicate removal: {len(crop_merge_ready)}')
print(f'City rows before: {city_before}, after duplicate removal: {len(city_merge_ready)}')

print('Any remaining duplicate rows in crop?', crop_merge_ready.duplicated().sum())
print('Any remaining duplicate rows in city?', city_merge_ready.duplicated().sum())

```

State-name variants found in crop_production:

message

0 No multi-spelling state-name variants found after deduplication.

Crop rows before: 242361, after duplicate removal: 242361

City rows before: 24850, after duplicate removal: 24850

Any remaining duplicate rows in crop? 0

Any remaining duplicate rows in city? 0

Task 4. AQI distribution shape

A histogram with a density curve shows where AQI values cluster, while a boxplot makes the spread and extreme values easy to see. Together they answer whether most cities sit in the middle of the AQI scale and whether a few very high readings are pulling the average upward.

```

In [8]: aqi = pd.to_numeric(city_merge_ready['AQI'], errors='coerce').dropna()

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.histplot(aqi, bins=40, kde=True, ax=axes[0], color='#1f77b4')
axes[0].set_title('AQI Distribution in city_day')

```

```

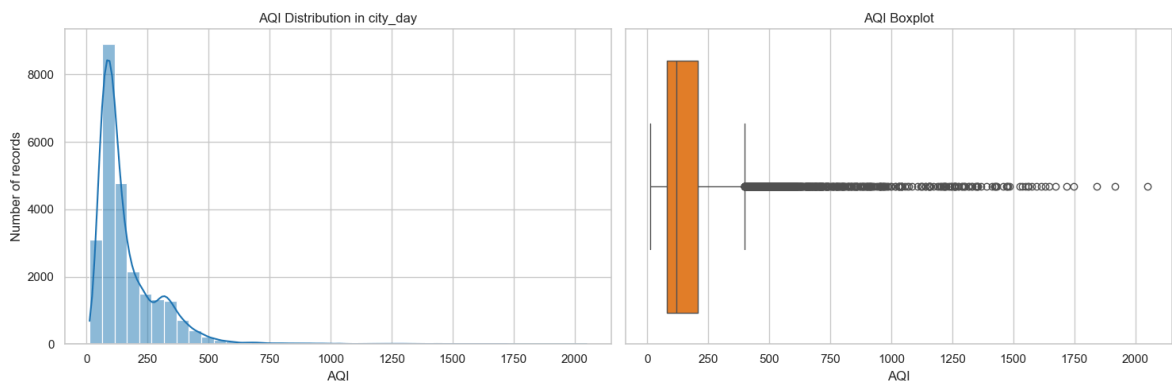
axes[0].set_xlabel('AQI')
axes[0].set_ylabel('Number of records')

sns.boxplot(x=aqi, ax=axes[1], color='#ff7f0e')
axes[1].set_title('AQI Boxplot')
axes[1].set_xlabel('AQI')

plt.tight_layout()
plt.show()

print('AQI summary:')
display(aqi.describe().to_frame('AQI'))
print('Skewness:', round(aqi.skew(), 3))
print('Observation 1: The histogram is right-skewed, so the average AQI is pulled')
print('Observation 2: The boxplot shows several extreme readings, so the median

```



AQI summary:

AQI	
count	24850.000000
mean	166.463581
std	140.696585
min	13.000000
25%	81.000000
50%	118.000000
75%	208.000000
max	2049.000000

Skewness: 3.397

Observation 1: The histogram is right-skewed, so the average AQI is pulled upward by a long high-value tail.

Observation 2: The boxplot shows several extreme readings, so the median is a safer measure of typical city AQI than the mean.

Task 5. Extreme AQI values

For AQI, values above 500 are implausible on the public scale, so capping is better than deleting rows. Capping preserves the record count while preventing a few extreme readings from dominating the summary statistics and the model.

```

In [9]: aqi_before = pd.to_numeric(city_merge_ready['AQI'], errors='coerce').dropna()
        extreme_mask = aqi_before > 500
        extreme_count = int(extreme_mask.sum())

        print('Method used to detect extremes: domain cap check at AQI > 500, which is a
        print('Extreme values found:', extreme_count)
        print('Maximum AQI before treatment:', float(aqi_before.max()))
        print('Mean AQI before treatment:', round(float(aqi_before.mean()), 2))

        city_outlier_clean = city_merge_ready.copy()
        city_outlier_clean['AQI'] = pd.to_numeric(city_outlier_clean['AQI'], errors='coe

        print('Maximum AQI after treatment:', float(city_outlier_clean['AQI'].max()))
        print('Mean AQI after treatment:', round(float(city_outlier_clean['AQI'].mean()))

        fig, axes = plt.subplots(2, 2, figsize=(15, 10))
        sns.histplot(aqi_before, bins=40, kde=True, ax=axes[0, 0], color='#d62728')
        axes[0, 0].set_title('AQI Before Treatment')
        axes[0, 0].set_xlabel('AQI')
        axes[0, 0].set_ylabel('Count')

        sns.histplot(city_outlier_clean['AQI'], bins=40, kde=True, ax=axes[0, 1], color=
        axes[0, 1].set_title('AQI After Treatment')
        axes[0, 1].set_xlabel('AQI')
        axes[0, 1].set_ylabel('Count')

        sns.boxplot(x=aqi_before, ax=axes[1, 0], color='#d62728')
        axes[1, 0].set_title('Boxplot Before Treatment')
        axes[1, 0].set_xlabel('AQI')

        sns.boxplot(x=city_outlier_clean['AQI'], ax=axes[1, 1], color='#2ca02c')
        axes[1, 1].set_title('Boxplot After Treatment')
        axes[1, 1].set_xlabel('AQI')

        plt.tight_layout()
        plt.show()

        print('Treatment applied: AQI values above 500 were capped at 500 instead of del
        print('Why this works: it preserves the number of observations while removing im

```

Method used to detect extremes: domain cap check at AQI > 500, which is above the valid public AQI range.

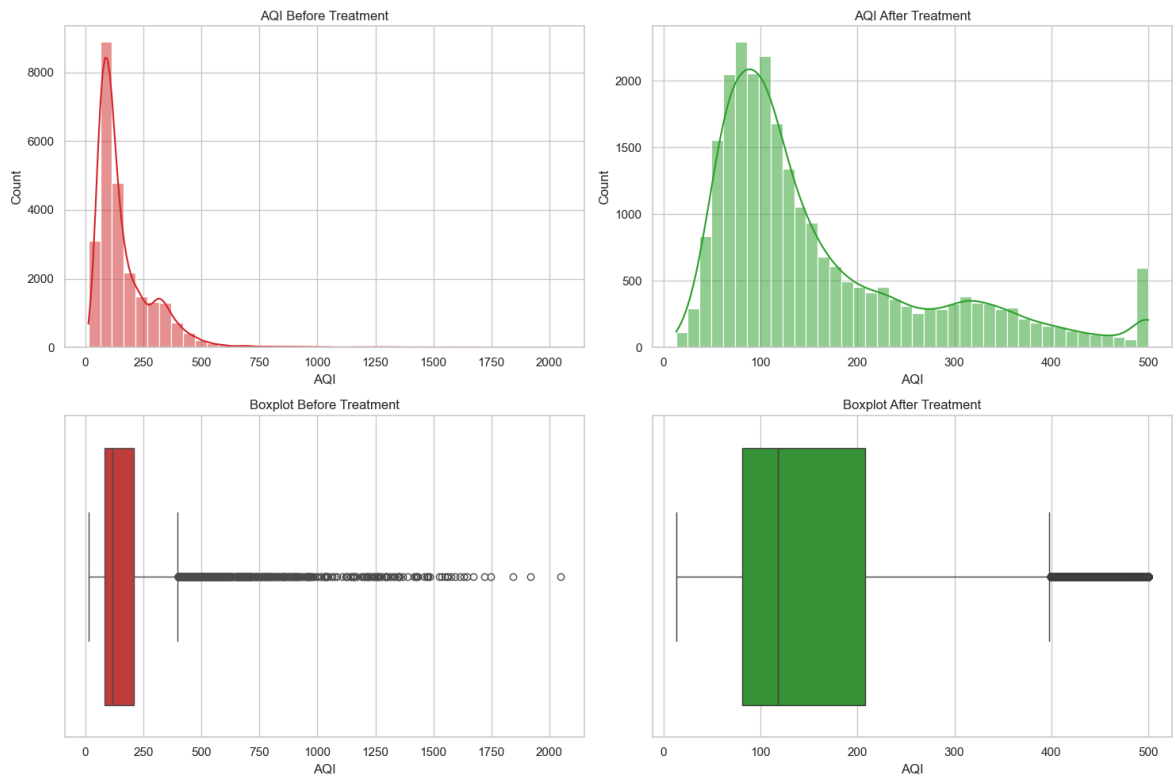
Extreme values found: 543

Maximum AQI before treatment: 2049.0

Mean AQI before treatment: 166.46

Maximum AQI after treatment: 500.0

Mean AQI after treatment: 160.92



Treatment applied: AQI values above 500 were capped at 500 instead of deleting the rows.

Why this works: it preserves the number of observations while removing impossible magnitudes that would distort the mean and spread.

Final outputs

After these steps, `crop_merge_ready` and `city_outlier_clean` are the cleaned versions to use later in the lab. The notebook keeps the original data untouched and documents each decision so the preprocessing is reproducible.

```
In [10]: print('Final crop shape:', crop_merge_ready.shape)
print('Final city shape after missing-value and duplicate handling:', city_outli
print('Final crop columns:', crop_merge_ready.columns.tolist())
print('Final city columns:', city_outlier_clean.columns.tolist())
```

Final crop shape: (242361, 7)

Final city shape after missing-value and duplicate handling: (24850, 11)

Final crop columns: ['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop', 'Area', 'Production']

Final city columns: ['City', 'Date', 'PM2.5', 'NO', 'NO2', 'NOx', 'CO', 'SO2', 'O3', 'Benzene', 'AQI']